Personalized Networking Assistant

Project Description

Personalized Networking Assistant is an AI-powered web application designed to help students, professionals, entrepreneurs, and conference attendees prepare for networking events by generating personalized and context-aware conversation starters. The application analyzes event descriptions using the DistilBERT model to extract important themes and combines them with the user's profile to generate meaningful conversation suggestions using the GPT-2 language model. To improve the reliability of AI-generated content, the application integrates the Wikipedia API to verify facts related to generated responses before presenting them to the user. The system also maintains networking session history, stores user profiles, logs system activities, and enables users to revisit previous conversations for future networking opportunities.

Developed using FastAPI for backend services and Streamlit for the frontend interface, the application provides a simple, responsive, and interactive user experience. By combining Natural Language Processing (NLP), Generative AI, and fact verification, the Personalized Networking Assistant enables users to communicate more confidently, prepare efficiently for networking events, and build stronger professional relationships.

Scenarios

Scenario 1: Professional Attending a Networking Event

A professional attending a networking event wants to start meaningful conversations with new people but struggles to think of relevant discussion topics. The **Personalized Networking Assistant** analyzes the event description and generates personalized conversation starters, helping the user communicate confidently and build valuable professional connections.

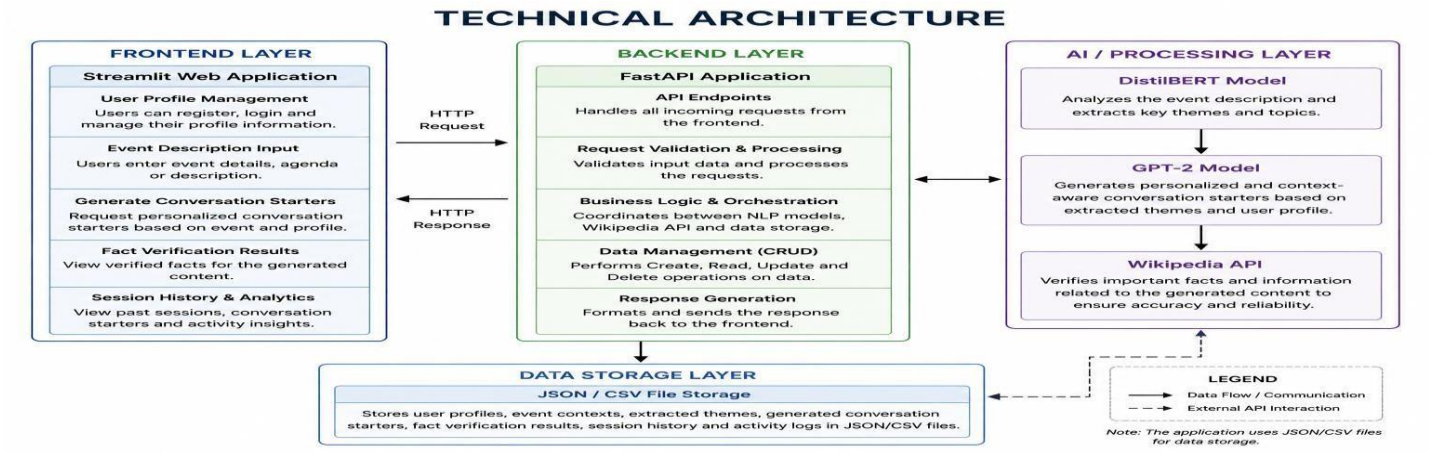
Scenario 2: Student Attending Workshops and Career Fairs

A student participating in a workshop or career fair wants to build professional relationships with industry experts but finds it difficult to personalize conversations. The application generates context-aware conversation starters based on the student's profile and event details, making interactions more natural and engaging.

Scenario 3: Conference Attendee Preparing for an Event

A conference attendee wants to verify technical facts before discussing them with others but lacks time to research reliable information. The **Personalized Networking Assistant** verifies important facts using the **Wikipedia API** and provides accurate, AI-generated conversation suggestions for confident networking.

Technical Architecture:



Description

The Personalized Networking Assistant utilizes a modular three-tier architecture to provide intelligent and personalized networking assistance. The frontend is developed using Streamlit, offering an interactive and user-friendly interface for entering user profiles and event descriptions. The FastAPI backend processes user requests, manages business logic, and coordinates communication between AI models and external services.

The application integrates the DistilBERT model to analyze event descriptions and extract key themes, while the GPT-2 model generates personalized and context-aware conversation starters based on the extracted themes and user profile. To improve the reliability of generated content, the application communicates with the Wikipedia API to verify factual information before presenting it to users. User profiles, networking sessions, generated conversation starters, verification results, and activity logs are stored using JSON/CSV (or MongoDB, if deployed) for efficient retrieval and future reference

Pre-requisites:

- Python Programming Knowledge: Understanding Python fundamentals and objectoriented programming concepts.
- FastAPI Framework: Knowledge of FastAPI for developing RESTful APIs and handling backend services.
- Streamlit Framework: Familiarity with Streamlit for building interactive web applications.
- Natural Language Processing (NLP): Basic understanding of NLP concepts and transformer-based language models.
- Hugging Face Transformers: Experience using pretrained models such as DistilBERT and GPT-2 for AI-powered text processing and generation.
- Wikipedia API: Basic knowledge of integrating external APIs for fact verification.
- Git and GitHub: Familiarity with version control, repository management, and collaborative development.
- Development Environment: Python Virtual Environment (venv), Visual Studio Code, and required Python libraries installed through the requirements.txt file.

Prior Knowledge Required

Before developing the **Personalized Networking Assistant**, developers should possess fundamental knowledge of programming, web development, artificial intelligence, and software engineering concepts. Familiarity with the following technologies and concepts will help in understanding, implementing, and maintaining the application effectively.

- **Python Programming:** Understanding Python syntax, functions, object-oriented programming, and package management.
- **FastAPI Framework:** Knowledge of developing RESTful APIs, request handling, routing, and backend application development.
- **Streamlit Framework:** Familiarity with creating interactive web applications and user interfaces using Streamlit.
- **Natural Language Processing (NLP):** Basic understanding of text preprocessing, language models, and transformer-based architectures.
- **Transformer Models:** Knowledge of pretrained transformer models, particularly **DistilBERT** for event theme extraction and **GPT-2** for AI-powered text generation.
- **Wikipedia API:** Understanding of integrating external APIs for fact verification and retrieving reliable information.
- **Git & GitHub:** Familiarity with version control, repository management, and collaborative software development.
- **Development Tools:** Experience using Visual Studio Code, Python Virtual Environment (venv), and dependency management using requirements.txt.

Project Objectives

The primary objective of the Personalized Networking Assistant is to help users prepare for professional networking events by providing intelligent, personalized, and fact-verified conversation suggestions. The application combines Natural Language Processing (NLP), Generative AI, and fact verification techniques to improve users' networking confidence and communication skills.

Objectives

- Develop an AI-powered web application for networking assistance.
- Analyze event descriptions using the DistilBERT model.
- Generate personalized conversation starters using GPT-2.
- Verify factual information using the Wikipedia API.
- Maintain user profiles and networking session history.
- Store conversation history and activity logs for future reference.
- Provide a responsive and user-friendly interface using Streamlit.
- Build a modular and scalable architecture using FastAPI.

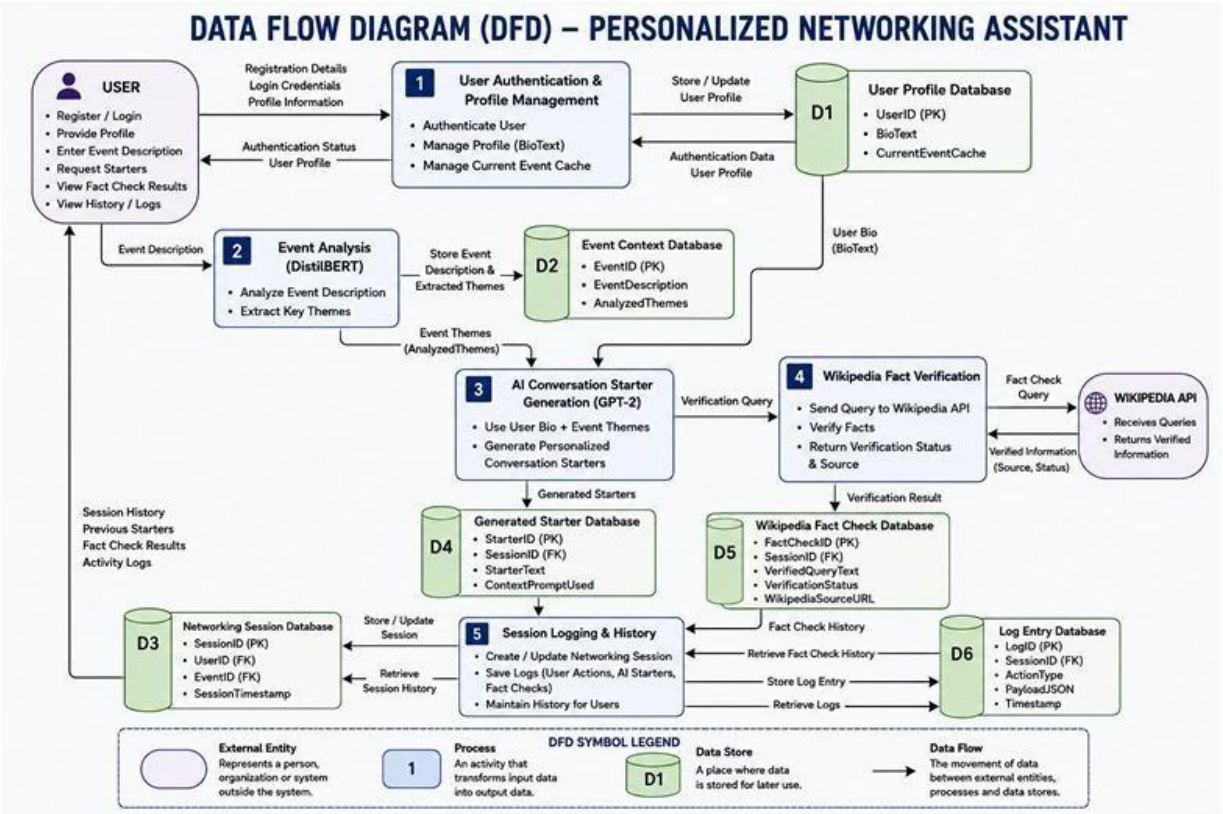
System Workflow

The workflow of the Personalized Networking Assistant consists of multiple stages that work together to generate intelligent conversation starters and maintain networking history.

Workflow Steps

1. User Registration and Login
2. User Profile Creation or Update
3. Event Description Submission
4. Event Theme Extraction using DistilBERT
5. Personalized Conversation Generation using GPT-2
6. Fact Verification through the Wikipedia API
7. Networking Session Storage
8. Activity Logging
9. Session History Retrieval

The application begins by authenticating the user and collecting profile information. Once the user provides an event description, the DistilBERT model extracts key themes and topics. These themes, combined with the user's profile, are processed by the GPT-2 model to generate personalized conversation starters. Before presenting the generated content, important facts are verified using the Wikipedia API to ensure reliability. Finally, all networking sessions, generated conversation starters, verification results, and system logs are stored, enabling users to review previous interactions and continuously improve their networking experience.



Milestone 1: Requirement Analysis & System Design

- The first milestone focused on understanding the project requirements and designing the overall system architecture for the Personalized Networking Assistant. During this phase, user needs, business objectives, and technical requirements were carefully analyzed to ensure that the proposed solution effectively addresses the challenges faced during professional networking events.
- A systematic requirement analysis process was followed to identify the problems experienced by students, professionals, and conference attendees while initiating conversations and preparing for networking opportunities. Based on these findings, the functional and non-functional requirements were defined, and the overall architecture of the application was designed. This milestone established a strong foundation for the successful implementation of the project.

Requirement Analysis

- The requirement analysis phase involved gathering user requirements through brainstorming sessions, empathy mapping, customer journey mapping, and problem statement analysis. These activities helped identify the major challenges faced by users, including difficulty in starting conversations, lack of personalized networking guidance, and limited access to reliable information before interacting with others.
- The analysis concluded that an AI-powered assistant capable of analyzing event descriptions, generating personalized conversation starters, and verifying factual information would significantly improve the networking experience for users.

System Design

- After finalizing the requirements, the overall system architecture was designed to define how different components of the application interact with one another. The architecture follows a modular approach consisting of the frontend, backend, AI processing modules, external API integration, and data storage.
- The frontend was designed using Streamlit to provide an interactive user interface, while FastAPI was selected as the backend framework for handling application logic and API communication. The DistilBERT model was integrated to analyze event descriptions and extract key themes, whereas GPT-2 was incorporated to generate personalized conversation starters. The Wikipedia API was integrated to verify facts before presenting AI-generated content to users. Networking sessions, user profiles, conversation history, and activity logs are stored in the application's data storage layer for future reference.

Milestone 2: Environment Setup & Initial Configuration

Development Environment Configuration

The development environment was configured by installing all the required software packages and libraries listed in the project's requirements.txt file. These dependencies provided the necessary functionality for AI processing, web development, API communication, and data handling. The major software components configured during this phase include:

- Python 3.x
- FastAPI
- Streamlit
- Hugging Face Transformers
- PyTorch
- Wikipedia API
- Pandas
- NumPy
- Uvicorn
- Requests
- Git & GitHub
- Visual Studio Code

AI Model Configuration

After installing the required libraries, the AI models were configured for their respective tasks. The DistilBERT model was integrated to analyze event descriptions and extract meaningful themes. These extracted themes provide contextual information that helps the system understand the focus of the networking event.

The GPT-2 language model was then configured to generate personalized conversation starters based on the extracted themes and user profile information. Proper prompt formatting and model initialization ensured that the generated responses were relevant, engaging, and context-aware.

To enhance the reliability of AI-generated content, the Wikipedia API was integrated for fact verification. The application verifies important information before displaying conversation suggestions to the user, improving trust and reducing the chances of presenting inaccurate information.

Project Repository Setup

The project repository was created using GitHub, enabling version control and collaborative development. The repository was organized into separate modules for frontend development, backend APIs, AI processing, fact verification, and utility functions. This modular structure improves code readability, simplifies maintenance, and supports future enhancements.

Environment Setup Outcome

At the completion of this milestone:

- The development environment was successfully configured.
- All required software packages and libraries were installed.
- FastAPI backend services were initialized.
- Streamlit frontend was configured.
- DistilBERT and GPT-2 models were integrated.
- Wikipedia API connectivity was established.
- GitHub repository structure was finalized.
- The application was ready for core system development.

Milestone 3: Core System Development

User Profile Management

The User Profile module was developed to collect and manage user information required for generating personalized conversation starters. Users can create and update their profiles by providing details such as their name, professional background, interests, and career goals. This information is used to tailor AI-generated responses according to the user's networking preferences.

The profile information is validated before processing and is maintained throughout the networking session to provide consistent and personalized recommendations.

Event Description Processing

The Event Description module enables users to provide information about the networking event they plan to attend. Users can enter details such as the event title, description, agenda, or discussion topics through the Streamlit interface.

Once submitted, the event description is sent to the FastAPI backend, where it is preprocessed by removing unnecessary characters and preparing the text for AI analysis. This preprocessing step improves the quality and accuracy of theme extraction.

Event Theme Extraction using DistilBERT

After preprocessing, the event description is analyzed using the **DistilBERT** transformer model. The model extracts key themes, important keywords, and contextual information that represent the primary focus of the event.

These extracted themes help the application understand the networking context and provide meaningful input for the conversation generation module. The use of DistilBERT enables efficient and accurate text analysis while maintaining fast response times.

Backend API Development

The backend of the application was implemented using **FastAPI**, which serves as the communication layer between the frontend and the AI modules. REST APIs were developed to handle user requests, process event descriptions, invoke AI models, and return results to the user.

The backend also manages user sessions, coordinates data flow between different components, and ensures secure communication throughout the application. Its modular design allows individual components to be updated or extended without affecting the overall system.

Features Implemented

The following core functionalities were completed during this phase:

- User Profile Creation and Management
- Event Description Submission
- Text Preprocessing
- Theme Extraction using DistilBERT
- FastAPI REST API Development
- Request Validation and Error Handling
- Communication between Frontend and Backend

Personalized Conversation Generation using GPT-2

The conversation generation module was developed using the **GPT-2** language model. After receiving the extracted themes from the **DistilBERT** model, the system combines them with the user's profile information to create a contextual prompt. This prompt is processed by GPT-2 to generate multiple personalized conversation starters that are relevant to the networking event and aligned with the user's interests.

Users can review the generated suggestions and request additional conversation starters whenever required. This functionality enables users to prepare for different networking situations and communicate with greater confidence.

Wikipedia Fact Verification

To improve the reliability of AI-generated content, the application integrates the **Wikipedia API** for fact verification. Important technical terms, concepts, and topics identified within the generated conversation starters are verified using reliable information available on Wikipedia. The verification results are presented to the user along with the conversation suggestions, helping users avoid sharing inaccurate information during professional discussions. This feature increases user confidence and enhances the credibility of AI-generated recommendations.

Networking Session Management

The Networking Session module records every interaction performed by the user during the application session. It stores information such as user profile details, event descriptions, extracted themes, generated conversation starters, verification results, and session timestamps. Maintaining networking session history allows users to revisit previous conversations, review successful discussion points, and continuously improve their networking skills for future events.

Activity Logging

A dedicated logging module was implemented to maintain records of important system activities and user interactions. The logging mechanism captures application events, API requests, processing status, and error messages, providing valuable information for debugging, monitoring, and performance analysis.

The logging system improves application maintainability by enabling developers to identify issues quickly and monitor system performance during execution.

Frontend Integration

After implementing the backend services and AI modules, all components were integrated into the **Streamlit** frontend. Users can seamlessly interact with the application by entering event descriptions, viewing extracted themes, generating conversation starters, verifying facts, and reviewing networking history through a simple and intuitive interface.

The frontend communicates with the FastAPI backend through REST APIs, ensuring smooth data exchange and real-time response generation.

Features Implemented

The following functionalities were successfully completed during this phase:

- AI-powered conversation starter generation
- GPT-2 model integration
- Wikipedia API-based fact verification
- Networking session management
- Activity logging
- Conversation history management
- Frontend and backend integration
- Interactive Streamlit user interface

Milestone 4: Integration & Optimization

After completing the development of all core modules, the next phase focused on integrating the individual components into a unified application and optimizing the system for improved performance, usability, and reliability. During this milestone, the frontend, backend, AI models,

and external services were successfully connected to ensure seamless communication and efficient data processing.

The integration process involved connecting the **Streamlit** frontend with the **FastAPI** backend through REST APIs. User requests submitted through the web interface were processed by the backend, which coordinated communication with the **DistilBERT** model for event analysis, the **GPT-2** model for conversation generation, and the **Wikipedia API** for fact verification. The processed results were then returned to the frontend and displayed in an organized and userfriendly manner.

Frontend and Backend Integration

The Streamlit frontend was integrated with the FastAPI backend to enable smooth interaction between users and the application. API endpoints were developed to receive user inputs, process requests, and return AI-generated responses in real time. This integration ensured efficient communication between the user interface and backend services while maintaining a responsive user experience.

AI Model Integration

The **DistilBERT** and **GPT-2** models were integrated into a single processing pipeline. Event descriptions submitted by users were first analyzed using DistilBERT to extract meaningful themes. These themes, along with the user's profile information, were then passed to GPT-2 to generate personalized conversation starters. This sequential integration improved the relevance and quality of AI-generated responses.

External API Integration

The **Wikipedia API** was integrated into the application to validate important concepts and technical terms present in the generated conversation starters. Fact verification results were displayed alongside the AI-generated suggestions, helping users gain confidence in the information before using it during networking events.

Performance Optimization

Several optimization techniques were implemented to improve application performance and responsiveness. Input validation was introduced to reduce processing errors, while efficient request handling minimized response time. Reusable functions and modular code organization improved maintainability and simplified future enhancements. Error handling mechanisms were also implemented to ensure the application continued functioning smoothly even when unexpected situations occurred.

User Interface Enhancement

The Streamlit interface was refined to provide a clean, intuitive, and user-friendly experience. Input forms, result displays, navigation components, and session history were organized logically to improve usability. Additional interface improvements included responsive layouts, clear labels, and structured presentation of generated conversation starters and verification results.

Features Achieved

During this milestone, the following improvements were successfully implemented:

- Integration of Streamlit frontend with FastAPI backend.
- Integration of DistilBERT and GPT-2 into a unified AI pipeline.
- Wikipedia API integration for real-time fact verification.
- Improved API communication and response handling.
- Enhanced error handling and input validation.
- Optimized application performance and responsiveness.
- Improved user interface design and navigation.

Milestone 5: Testing & Validation

The final milestone focused on evaluating the performance, functionality, and reliability of the **Personalized Networking Assistant** through comprehensive testing and validation. The objective of this phase was to ensure that all modules operated correctly, interacted seamlessly, and met the functional and non-functional requirements defined during the planning stage.
Different testing methodologies were applied to verify the correctness of the application, identify potential issues, and improve the overall user experience. Each module was tested individually before performing integration testing on the complete system. The application was also validated using real-world networking scenarios to ensure that the generated conversation starters were relevant, context-aware, and factually reliable.

Functional Testing

Functional testing was performed to verify that each feature of the application operated according to the specified requirements. The testing covered user profile management, event description processing, AI-based conversation generation, Wikipedia fact verification, networking session management, and activity logging.
The system successfully accepted user inputs, analyzed event descriptions, generated personalized conversation starters, verified factual information, and stored session details without any major functional issues.

Integration Testing

Integration testing was conducted to ensure smooth communication between the frontend, backend, AI models, and external services. The Streamlit frontend was successfully integrated with the FastAPI backend, while the DistilBERT and GPT-2 models interacted efficiently to process event descriptions and generate personalized conversation suggestions. The Wikipedia API also responded correctly to fact verification requests.

Performance Testing

Performance testing evaluated the application's responsiveness and execution efficiency. The system demonstrated stable performance while processing user requests, generating AI-powered conversation starters, and retrieving verification results. Optimized request handling and modular implementation contributed to reduced response time and improved overall performance.

User Validation

The application was validated using different networking scenarios involving professionals, students, and conference attendees. The generated conversation starters were reviewed to ensure that they were relevant to the event context and aligned with the user's profile. Fact verification through the Wikipedia API further improved the reliability of AI-generated content, making the application suitable for real-world networking situations.

Testing Summary

The following testing activities were successfully completed:

- User Profile Validation
- Event Description Processing
- DistilBERT Theme Extraction
- GPT-2 Conversation Generation
- Wikipedia Fact Verification
- Networking Session Management
- Activity Logging
- Frontend–Backend Communication
- Overall System Validation

Validation Results

The testing process confirmed that the **Personalized Networking Assistant** achieved its intended objectives. The application successfully generated personalized conversation starters, analyzed event descriptions accurately, verified factual information, maintained networking sessions, and provided an intuitive user experience. No critical functional errors were observed during the final validation phase, demonstrating the stability and reliability of the implemented system.

Deployment

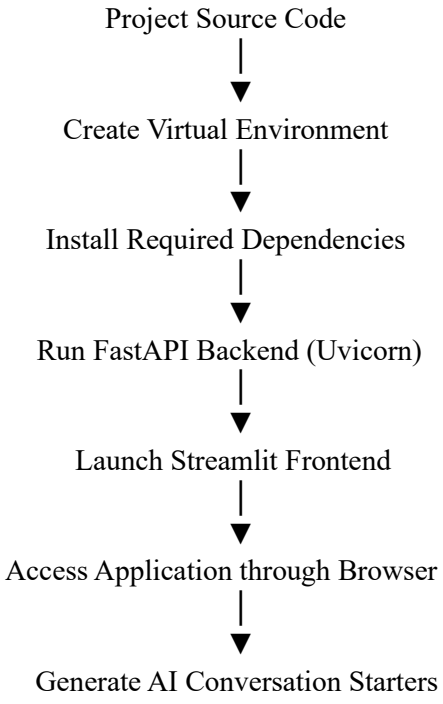
After successful development, integration, and testing, the **Personalized Networking Assistant** was prepared for deployment. The deployment phase focused on making the application accessible to users by configuring the runtime environment and launching both the frontend and backend services. The application was designed to support local deployment and can be extended for cloud deployment in future versions.

The deployment process begins by creating a Python virtual environment and installing all required dependencies listed in the **requirements.txt** file. Once the environment is configured, the FastAPI backend is launched using the **Uvicorn** server, while the Streamlit frontend is started separately. The frontend communicates with the backend through REST APIs to process user requests and display AI-generated results.

Local Deployment

1. Clone the project repository from GitHub.
2. Create and activate a Python virtual environment.
3. Install all required dependencies using the requirements.txt file.
4. Start the FastAPI backend using the Uvicorn server.
5. Launch the Streamlit frontend.
6. Open the application in a web browser and begin using the Personalized Networking Assistant.

Deployment Workflow



Deployment Environment

The application was developed and deployed using the following software environment:

Component	Description
Operating System	Windows 11
Programming Language	Python 3.x
Backend Framework	FastAPI
Frontend Framework	Streamlit
AI Models	DistilBERT, GPT-2
API Integration	Wikipedia API
Development Environment	Visual Studio Code
Version Control	Git & GitHub
Package Manager	pip
Runtime Server	Uvicorn

Project Structure

The **Personalized Networking Assistant** follows a modular full-stack architecture that separates the frontend, backend, AI processing, and configuration files into independent modules. This modular design improves maintainability, scalability, and code readability while allowing each component to perform a dedicated responsibility. The project is developed using **React with TypeScript and Vite** for the frontend and **Express (Node.js)** for backend services. AI-powered conversation generation is implemented using the **Google GenAI SDK (Gemini 3.5 Flash)**, while factual verification is performed using the **Wikipedia REST API**.

Project Directory Structure

Personalized-Networking-Assistant/

```
│
├── src/
│   ├── App.tsx
│   ├── main.tsx
│   ├── index.css
│   └── types.ts
│
├── server.ts
├── package.json
├── tsconfig.json
├── vite.config.ts
```

```
├── index.html
├── README.md
├── .env.example
└── .gitignore
```

This directory structure separates the client-side interface, backend services, configuration files, and project documentation, making the application easier to develop, test, and maintain.

Description of Project Files src/App.tsx

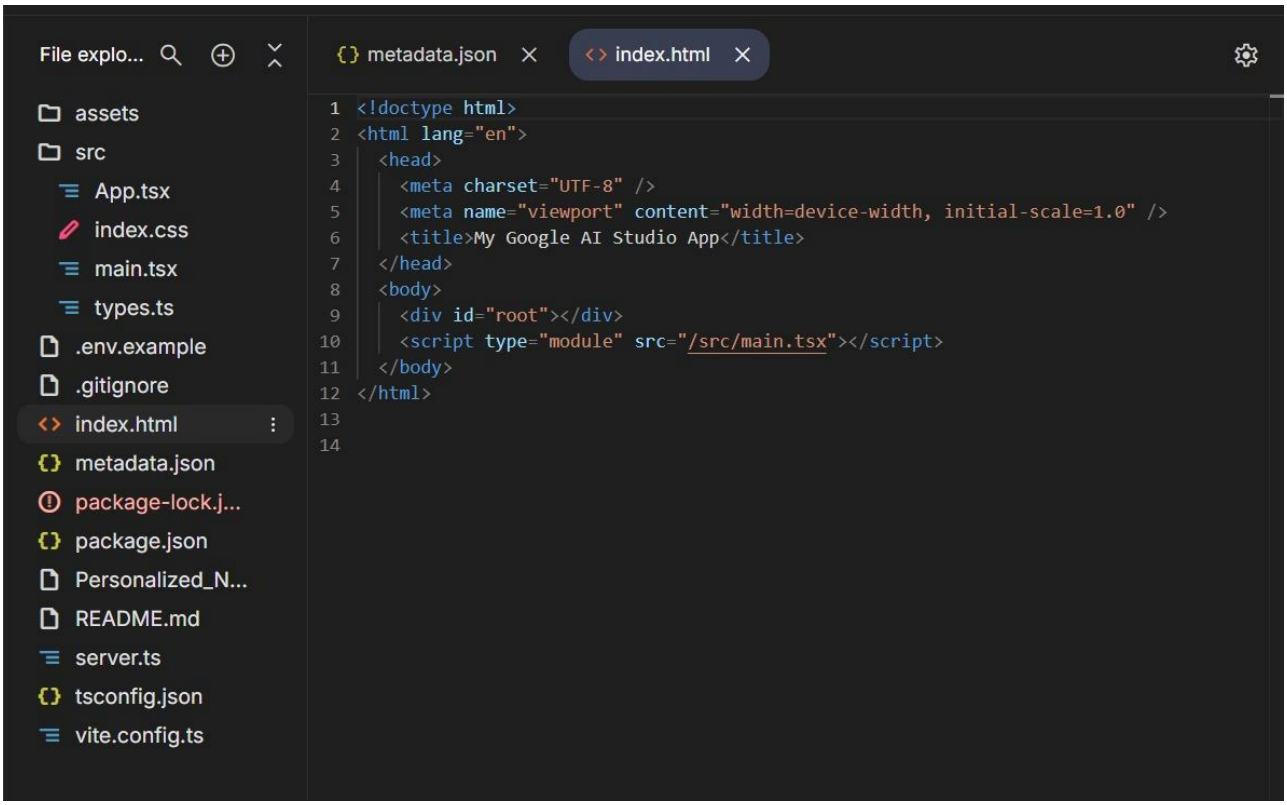
This file contains the main React application and implements the complete user interface. It manages user profiles, event information, AI-generated conversation starters, fact verification, networking history, and user feedback. It also controls navigation between the Workspace, Fact Check Hub, and Saved Strategies sections.

src/main.tsx

This is the entry point of the React application. It renders the main application component and loads the global stylesheet required for the user interface.

index.html

This is the root HTML file that initializes the React application by providing the root container where the frontend is rendered.



src/index.css
This file contains the application's global styling, typography settings, animations, and Tailwind CSS configuration. It ensures a modern, responsive, and consistent user interface throughout the application.

src/types.ts
This file defines the TypeScript interfaces used throughout the application, including conversation starters, networking history, fact verification results, and feedback logs. These interfaces improve type safety and maintain consistency across different modules.

server.ts
The **server.ts** file serves as the backend of the application using **Express.js**. It implements REST API endpoints for event analysis, conversation generation, fact verification, networking history management, and user feedback logging. The backend also integrates the Google GenAI SDK and Wikipedia API while managing local JSON-based data storage.

package.json
This file manages the project's dependencies, scripts, and build configuration. It defines the required libraries such as React, Express, Vite, Tailwind CSS, TypeScript, and the Google GenAI SDK, along with development and production commands.

tsconfig.json
This configuration file specifies the TypeScript compiler settings, module resolution, target JavaScript version, and JSX support used throughout the project.

vite.config.ts
This file configures the Vite build tool, React plugin, Tailwind CSS integration, path aliases, and development server settings for the frontend application.

README.md
The README file provides an overview of the project, installation steps, technology stack, API endpoints, project structure, and instructions for running the application in development and production environments.

.env.example
This file contains the template for environment variables such as the **Gemini API Key** and application URL. It helps developers configure the application securely without exposing sensitive credentials.

Results

The **Personalized Networking Assistant** was successfully developed and implemented as an AIpowered web application that assists users in preparing for professional networking events. The application integrates Artificial Intelligence, Natural Language Processing, and fact verification techniques to generate personalized conversation starters, verify technical concepts, and maintain networking history.
The system was evaluated by testing its core functionalities under different networking scenarios, including professional networking events, career fairs, and technical conferences. The application demonstrated reliable performance and successfully generated context-aware conversation starters based on user profiles and event descriptions.

Achieved Results

The following objectives were successfully accomplished during the implementation of the project:

- Successfully developed a responsive web application using **React**, **TypeScript**, and **Vite**.
- Implemented a secure backend using **Express.js** to process user requests and manage API communication.
- Integrated the **Google GenAI SDK (Gemini 3.5 Flash)** to generate personalized and context-aware conversation starters.
- Integrated the **Wikipedia REST API** to verify technical concepts and provide reliable information before networking interactions.
- Developed user profile management to personalize AI-generated recommendations.
- Implemented networking history management to allow users to review previous event preparations and conversation strategies.
- Added a feedback mechanism for evaluating AI-generated conversation starters and improving future recommendations.
- Successfully integrated frontend, backend, AI services, and external APIs into a single fullstack application.

Functional Results

The completed application provides the following functionalities:

Module	Result
User Profile Management	Successfully stores user background and interests.
Event Analysis	Analyzes event descriptions and identifies important discussion themes.
Conversation Generation	Generates personalized conversation starters using AI.
Fact Verification	Retrieves reliable summaries using the Wikipedia REST API with AI support.
Networking History	Stores previous networking sessions for future reference.

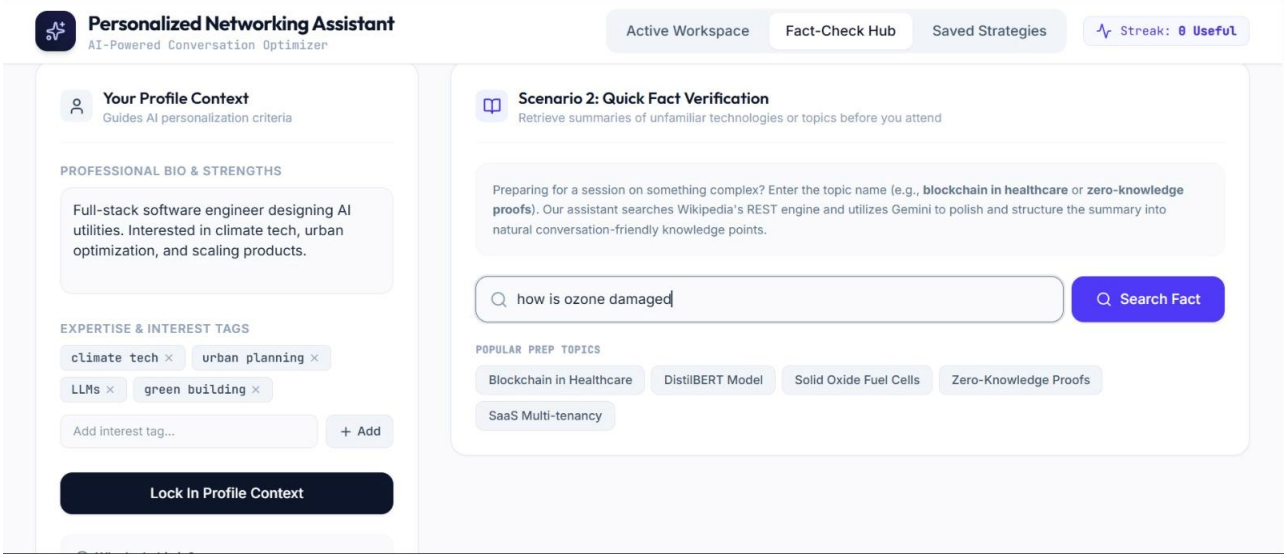
Module	Result
Feedback Logging	Records user feedback to evaluate AI-generated suggestions.
Interactive Dashboard	Provides an intuitive interface for managing networking activities.

Application Outputs

The developed application consists of the following major interfaces:

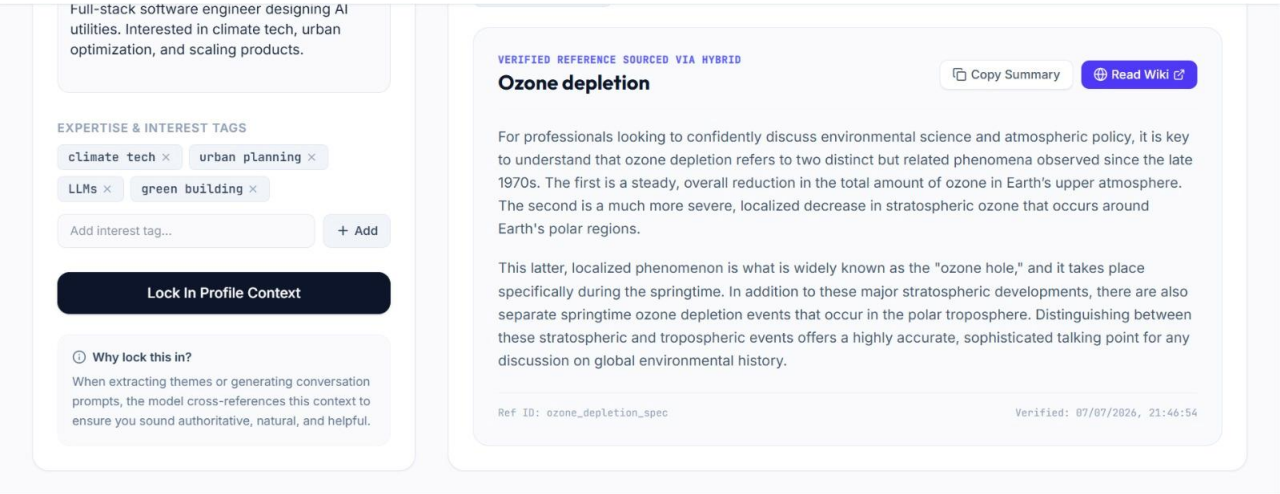
- Home Dashboard
- User Profile Management
- Event Context Input
- AI-Generated Conversation Starters
- Quick Fact Verification Hub
- Saved Networking Strategies
- Feedback Management

These interfaces work together to provide a complete networking preparation platform with an intuitive and user-friendly experience.



Performance Evaluation

The application demonstrated stable performance during testing. User requests were processed efficiently, and AI-generated responses were delivered with minimal delay. The integration between the React frontend, Express backend, Google GenAI SDK, and Wikipedia REST API functioned smoothly, ensuring reliable communication between all system components.



Advantages & Limitations

Advantages

1. Personalized Conversation Generation

The application generates conversation starters based on the user's profile, interests, and event description, enabling more natural and meaningful interactions during networking events.

2. AI-Powered Event Analysis

The system analyzes event descriptions using Artificial Intelligence to identify important discussion topics and networking opportunities, allowing users to prepare effectively before attending an event.

3. Reliable Fact Verification

The integration of the Wikipedia REST API enables users to verify technical concepts and unfamiliar topics before engaging in discussions, improving confidence and reducing misinformation.

4. Interactive User Interface

The React-based user interface provides an intuitive and responsive experience, allowing users to easily manage profiles, analyze events, generate conversation starters, and review networking history.

5. Networking History Management
The application maintains records of previous networking sessions, enabling users to revisit conversation strategies, review feedback, and improve their networking skills over time.

6. Modular Architecture
The separation of frontend, backend, AI services, and configuration files improves maintainability, scalability, and future extensibility of the application.

7. Easy Deployment
The project can be deployed locally using Node.js and can be extended to cloud platforms with minimal modifications, making it suitable for both development and production environments.

Limitations

1. Dependency on AI Responses
The quality of generated conversation starters depends on the responses produced by the AI model. Occasionally, suggestions may require minor user refinement to better suit a specific networking context.

2. Internet Connectivity Requirement
The application requires an active internet connection for AI response generation and Wikipedia fact verification. Limited connectivity may affect system functionality.

3. Dependence on User Input
The relevance of generated conversation starters depends on the quality and completeness of the user's profile information and event description.

4. Limited External Knowledge Sources
Currently, fact verification relies primarily on the Wikipedia REST API. Information from research papers, technical documentation, or other authoritative knowledge sources is not yet incorporated.

5. Local Data Storage
The current implementation stores networking history and feedback locally using JSON files. This approach is suitable for demonstration purposes but is not ideal for multi-user environments or large-scale deployments.

Conclusion

The **Personalized Networking Assistant** was successfully designed and developed as an AIpowered web application that assists users in preparing for professional networking events. The application combines **Artificial Intelligence**, **Natural Language Processing (NLP)**, and **fact verification** to generate personalized conversation starters based on user profiles and event descriptions. By leveraging modern web technologies and AI models, the system provides users with meaningful, context-aware suggestions that enhance confidence and improve networking experiences.

Throughout the project, a structured software development approach was followed, beginning with requirement analysis and system design, followed by environment setup, core system development, integration, optimization, testing, and deployment. The application was implemented using **React**, **TypeScript**, **Vite**, and **Express.js**, while the **Google GenAI SDK (Gemini)** was utilized for intelligent conversation generation and the **Wikipedia REST API** was integrated for reliable fact verification. This combination of technologies enabled the development of a responsive, scalable, and user-friendly application.

The completed system successfully supports user profile management, event analysis, AI-powered conversation generation, networking history management, and feedback collection. Its modular architecture simplifies future maintenance and allows additional features to be incorporated with minimal changes to the existing system. Comprehensive testing confirmed that all major functionalities operate correctly and meet the project objectives.
In conclusion, the **Personalized Networking Assistant** provides an innovative solution for individuals seeking to improve their networking skills and professional interactions. By combining AI-driven personalization with reliable information retrieval, the application enables users to communicate more confidently and effectively in networking events, career fairs, workshops, and conferences. With future enhancements such as multilingual support, cloud deployment, mobile applications, and professional platform integration, the system has significant potential to evolve into a comprehensive AI-based networking companion suitable for real-world applications.

Key Achievements

- Successfully developed an AI-powered networking assistant.
- Implemented a modern full-stack architecture using **React**, **TypeScript**, **Vite**, and **Express.js**.
- Integrated **Google GenAI (Gemini)** for personalized conversation generation.
- Integrated the **Wikipedia REST API** for factual information verification.
- Developed modules for user profile management, event analysis, networking history, and feedback logging.
- Designed a modular and scalable architecture to support future enhancements.
- Successfully tested and validated all core functionalities before deployment.